
Guía de Estudio por Capítulos

Infraestructura como Código (IaC) · Resumen del módulo en 12 capítulos

Capítulo 1 — Fundamentos de IaC y DevSecOps

La **Infraestructura como Código (IaC)** es la práctica de gestionar y aprovisionar infraestructura mediante archivos de configuración legibles por máquina, en lugar de procesos manuales. La infraestructura se trata como el código de software: vive en Git, es versionada, revisable y auditable, y garantiza que el entorno sea idéntico en Dev, Staging y Producción. El equipo deja de “hacer clic en la consola” para describir todo su entorno en archivos de texto.

La evolución va de lo **manual** a los **scripts** (IaC v1, frágil y no idempotente), al **declarativo** de hoy (se describe el estado deseado y la herramienta calcula el diff), y a un cuarto estadio con **IA**. El enfoque **declarativo** describe el QUÉ; el **imperativo**, el CÓMO. DevSecOps integra la seguridad en cada fase del ciclo: el **Shift-Left** lleva las validaciones de seguridad antes de aplicar a producción.

Puntos clave

- **Idempotencia:** ejecutar una operación N veces produce siempre el mismo resultado.
- **5 principios:** idempotencia; versionamiento (todo en Git); reproducibilidad (mismo código = misma infraestructura; elimina los servidores “snowflake” configurados a mano); modularidad (un módulo = una responsabilidad, con inputs y outputs claros); y legibilidad/trazabilidad.
- **Declarativo:** Terraform, CloudFormation, Bicep. **Imperativo:** Ansible, scripts Bash.
- **Shift-Left:** TFLint (lint), Checkov (análisis estático), GitLeaks (secretos), Trivy (containers), Terratest (tests).
- **Beneficios:** velocidad, consistencia, menos errores, auditoría, recuperación y escalabilidad.

Capítulo 2 — Herramientas y enfoques de IaC

Las herramientas líderes se diferencian por alcance, modelo y gestión de estado. **Terraform** es multi-cloud, declarativo, con estado externo (`.tfstate`) y lenguaje HCL. **CloudFormation** es solo AWS, con rollback automático y estado gestionado por AWS. **Ansible** es multi-cloud, imperativo, agentless y sin estado. **Pulumi** usa lenguajes de propósito general; **Azure Bicep** es solo Azure (sucesor de ARM); **Vagrant** sirve para VMs locales.

Una nueva categoría son las **herramientas de IA** (GitHub Copilot, Amazon Q Developer, ChatGPT/Claude), útiles para generar boilerplate, traducir entre herramientas y documentar, pero que requieren revisión humana en lo crítico.

Puntos clave

- **Guía de elección:** 100 % AWS + rollback → CloudFormation; multi-cloud/híbrido → Terraform; configurar software en servidores → Ansible; lenguajes generales → Pulumi; solo Azure → Bicep; entornos locales → Vagrant.
- **La IA** es ideal para boilerplate, traducción CF↔Terraform y documentación.
- **Revisión humana SIEMPRE** en: IAM/permisos, security groups, estado/backend, producción crítica y todo `terraform apply` en producción.

Capítulo 3 — Terraform: modelo mental y flujo de trabajo

HashiCorp se fundó en **2012** y lanzó Terraform 0.1 en **2014**. En **2023** cambió la licencia de OSS a **BSL**, y la comunidad creó **OpenTofu** como fork open source. Hoy Terraform es el estándar #1 multi-cloud. Su modelo mental: describes el **estado deseado**, Terraform calcula el **diff** con el estado actual, lo muestra en `plan`, y solo se aplica lo revisado. El estado se guarda en `terraform.tfstate` (JSON), su fuente de verdad.

Puntos clave

- `terraform init` — descarga providers, inicializa el backend, crea `.terraform/` y el lock file.
- `terraform plan` — calcula el diff; no hace cambios reales (`-out=plan.tfplan`).
- `terraform apply` — ejecuta el plan, pide confirmación (yes) y actualiza el `.tfstate`.
- `terraform destroy` — elimina todos los recursos del estado.
- **Comandos complementarios:** `terraform fmt` (formato consistente), `terraform validate` (sintaxis) y `terraform output` (consulta de outputs).
- **Símbolos del plan:** `+` crear · `~` modificar in-place · `-` destruir · `-/+` destruir y recrear.

Capítulo 4 — Terraform: estructura, estado y buenas prácticas

Un proyecto típico se organiza en cuatro archivos: `providers.tf` (versión y provider), `variables.tf` (type/description/default), `main.tf` (recursos y data sources) y `outputs.tf` (valores exportados). El `terraform.tfvars` sobrescribe defaults y va en `.gitignore`; el `.terraform.lock.hcl` sí se commitea. Las variables y outputs son el **contrato del módulo**, y el Registry ofrece más de 14 000 módulos públicos.

Para colaboración real se usa un **backend remoto** (S3 para el archivo + DynamoDB para el locking), y los **workspaces** separan ambientes. La idempotencia se ve en práctica: el primer `apply` crea los recursos; el segundo, sin cambios, reporta “No changes. Infrastructure is up-to-date.”.

Puntos clave

- `terraform.tfvars` va en `.gitignore`; `.terraform.lock.hcl` **sí** se commitea.
- **Backend remoto:** S3 (archivo de estado) + DynamoDB (lock distribuido).
- **Workspaces:** separan dev/staging/prod sobre el mismo código.
- **Prácticas:** SSH restringido con `/32`, AMI dinámica con `data "aws_ami"`, `user_data` para instalar software, `fmt+validate` antes del `plan`, IMDSv2 (`http_tokens = "required"`), `destroy` al terminar laboratorios.
- **Desventajas:** gestión del `.tfstate`, curva de HCL, no configura el software interno, licencia BSL (mitigada por OpenTofu) y lentitud en infra muy grande.

Capítulo 5 — AWS CloudFormation

CloudFormation es el servicio **nativo de AWS** que crea infraestructura a partir de una plantilla en YAML o JSON. Es declarativo: describes el QUÉ y AWS calcula el cómo. Un **stack** es el conjunto de recursos creados desde un template y se gestiona como una sola unidad. No usa archivo de estado externo (AWS gestiona el estado) y ofrece **rollback automático** ante errores.

Puntos clave

- **Resources** es la única sección **obligatoria** (Parameters, Outputs, Mappings y Conditions son opcionales).
- Soporta **Change Sets** (ver impacto antes de aplicar), **StackSets** (multi-cuenta y multi-región) y nested stacks.
- **AWS SAM** y **AWS CDK** generan plantillas de CloudFormation.
- **Elegir CF** para 100 % AWS, rollback garantizado y StackSets; **Terraform** para multi-cloud, HCL y registry de módulos.

Capítulo 6 — Ansible

Ansible es **agentless**: usa SSH para llegar a los nodos gestionados. Sus playbooks se escriben en YAML legible, dispone de más de 3 000 módulos en Ansible Galaxy, y logra idempotencia comparando el estado previo (`state: present/absent`). Sus componentes son el nodo de control, los nodos gestionados, el inventario, los módulos, las tasks, los playbooks y los roles; **Ansible Vault** cifra secretos.

Frente a Terraform es **complementario, no rival**: Terraform aprovisiona la infraestructura y Ansible configura el software dentro de los servidores.

Puntos clave

- **Agentless** vía SSH; **imperativo** y **sin estado**.
- Idempotencia mediante `state: present / absent`.
- **Patrón de clase**: Terraform aprovisiona la infra; Ansible configura el software interno.
- **Ansible Vault** para gestionar secretos cifrados.

Capítulo 7 — Servicios AWS aprovisionables con IaC

Todo servicio cloud encaja en **5 categorías**: Compute, Networking, Storage & Data, Identity y Observability. El módulo recorre los **9 servicios fundamentales**: VPC, ECS+Fargate, ECR, ALB, ElastiCache, Cloud Map, IAM, CloudWatch Logs y Security Groups; más servicios adicionales (S3, RDS/Aurora, Lambda, Route 53) y sus equivalencias en Azure y GCP.

Puntos clave

- **5 categorías universales**: Compute, Networking, Storage & Data, Identity, Observability.
- **Cloud Map** = service discovery por DNS interno; **Security Groups** = firewall a nivel de recurso; **IAM** = identidad con mínimo privilegio.
- **ECR** = registro privado de imágenes Docker; **ALB** = balanceador HTTP/HTTPS con health checks; **ElastiCache** = Redis/Memcached administrado; **CloudWatch Logs** = centralización de logs.
- **Multi-nube**: VPC ↔ VNet ↔ VPC · ECS Fargate ↔ Container Apps ↔ Cloud Run · ECR ↔ ACR ↔ Artifact Registry.

Capítulo 8 — Microservicios

Una arquitectura de microservicios divide la aplicación en servicios pequeños, autónomos y desplegables por separado, cada uno con una **única responsabilidad**. Son autónomos, dueños de su propia base de datos (**database-per-service**) y se comunican por la red: APIs ligeras (REST/gRPC) para lo síncrono y eventos para lo asíncrono.

Frente al **monolito**, ofrecen despliegue y escalado independientes y aislamiento de fallos, a cambio de complejidad de red, consistencia distribuida y más infraestructura. La regla práctica: empezar simple y dividir cuando el dominio y el equipo lo justifiquen.

Puntos clave

- **database-per-service**: cada servicio es dueño de su base de datos.
- **Brokers asíncronos**: Kafka, RabbitMQ, NATS, AWS SNS/SQS/EventBridge.
- **Patrones**: comunicación síncrona/asíncrona, service discovery y API Gateway (entrada única).
- **Ventaja clave**: escalar solo el servicio que lo necesita y aislar fallos.

Capítulo 9 — Kubernetes

Kubernetes (K8s) es el estándar **open source** para orquestar contenedores: automatiza despliegue, escalado, balanceo y auto-reparación, y es **portable** entre nubes y on-premises (evita el vendor lock-in). Su arquitectura incluye el Cluster, el Control Plane (API Server, Scheduler, Controller Manager y **etcd**), los Nodos (kubelet, kube-proxy), y objetos como Pod, Deployment, Service, Ingress, ConfigMap/Secret y Namespace.

En AWS, **EKS** es el Kubernetes administrado (AWS gestiona el control plane). El cluster se aprovisiona con IaC y los manifiestos se versionan en Git (GitOps con ArgoCD/Flux); Helm y Kustomize ayudan a parametrizar por entorno.

Puntos clave

- **Pod** = unidad mínima de despliegue; **etcd** = base clave-valor del estado del cluster.
- **Deployment** gestiona réplicas, rolling updates y rollbacks; **HPA** = escalado horizontal automático.

- **ECS vs EKS:** simplicidad + todo-AWS → ECS (Fargate); portabilidad + flexibilidad → Kubernetes (EKS).
- **Helm** = gestor de paquetes (charts); **Kustomize** = overlays por entorno.

Capítulo 10 — CI/CD: conceptos y GitHub Actions

La **Integración Continua** (Fowler) consiste en integrar el trabajo frecuentemente —al menos una vez al día— validando cada integración con una compilación automatizada y pruebas. La **Entrega Continua** mantiene el software liberable en todo momento; el **Despliegue Continuo** lleva cada cambio aprobado a producción sin intervención humana. En **GitOps**, el repositorio Git es la fuente de verdad y los cambios se aplican vía PRs revisados.

GitHub Actions es declarativo: un **Workflow** (YAML) contiene **Jobs** (en runners efímeros) con **Steps** que usan **Actions** del Marketplace. Maneja secrets, variables, outputs, concurrency, cache y matrix, con permisos por job y OIDC.

Puntos clave

- **CI** = integración frecuente + validación automática; **Continuous Deployment** = a producción sin intervención humana.
- **Triggers:** `push`, `pull_request`, `schedule`, `workflow_dispatch` (manual), `workflow_call` (reutilizable).
- Un **Job** corre en una VM efímera (runner); los **Secrets** se inyectan en runtime y no se loguean.
- **OIDC** con rol IAM temporal evita credenciales long-lived en el repo.

Capítulo 11 — Caso práctico: pipeline NestJS en ECS

Pipeline end-to-end con **GitHub Actions + Terraform** sobre un monorepo NestJS con dos microservicios (`orders` y `notifications`), comunicación por NATS y persistencia en Redis; la infraestructura (ECS Fargate + ALB + ECR + ElastiCache) se define en Terraform. El pipeline se separa en **tres workflows**: `ci.yml` (lint/build/test), `terraform-plan.yml` (plan + comentario en el PR) y `deploy.yml` (post-merge: infra + push de imágenes + rollout).

Puntos clave

- **Tres workflows:** `ci.yml` · `terraform-plan.yml` · `deploy.yml`.
- **Problema del huevo y la gallina:** se resuelve con `apply` en dos fases (primero los repos ECR, luego push de imágenes, después `apply` completo).
- **Image tag** = `github.sha`: cada deploy es rastreable; el rollback es redeployar un SHA anterior.
- **State remoto:** S3 (archivo) + DynamoDB (lock); el runner es efímero, por eso no puede ser local.

Capítulo 12 — Comparativa de CI/CD y síntesis final

Jenkins es un servidor self-hosted que ejecuta un Jenkinsfile (Groovy): más control pero también más operación; conviene para compliance estricto o redes air-gapped. **CircleCI** es un SaaS donde los “orbs” equivalen a las actions, con caching potente pero mayor costo. En la comparativa GitHub Actions vs Jenkins vs CircleCI, para el proyecto NestJS en ECS gana **GitHub Actions** por simplicidad y costo.

Síntesis del módulo: todo servicio cloud cae en 5 categorías; el mismo concepto cambia de nombre entre nubes; “administrado” significa que el proveedor hace el trabajo pesado; la seguridad es transversal; serverless escala y cobra por uso. La base del enfoque es **laC declarativo + idempotencia + estado gestionado + versionamiento en Git**, y CI/CD cierra el ciclo validando y aplicando cada cambio de forma auditable.

Puntos clave

- **Jenkins:** self-hosted, Groovy. **CircleCI:** SaaS, orbs. **Ganador del caso:** GitHub Actions.
- **Seguridad transversal:** IAM (quién) + Security Groups (qué tráfico) en todas las capas.
- **Base del módulo:** laC declarativo, idempotencia, estado gestionado y versionamiento en Git.

- Microservicios y Kubernetes añaden flexibilidad y complejidad: usar solo cuando el dominio y el equipo lo justifiquen.